HTML, CSS & Bootstrap Complete Interview Preparation Guide

From Scratch to Associate Software Engineer Level

Covering L1 & L2 Technical Interview Topics

■ Topics Covered
• Complete HTML Fundamentals & HTML5
• CSS from Basics to Advanced (Flexbox, Grid, Animations)
• Bootstrap 5 Framework
• 100+ Interview Questions with Answers
• Code Examples & Best Practices
• L1 (Basic) & L2 (Intermediate) Interview Topics

Table of Contents

PART 1: HTML FUNDAMENTALS

1.1 Introduction to HTML

HTML (HyperText Markup Language) is the standard markup language used to create web pages. It describes the structure of a web page using a series of elements and tags. HTML is not a programming language; it is a markup language that tells browsers how to display content.

Key Points:

- **HTML** stands for HyperText Markup Language
- It is the standard markup language for documents designed to be displayed in a web browser
- HTML describes the **structure** of a web page
- HTML consists of a series of **elements** that tell the browser how to display content
- The latest version is **HTML5**, which introduced new semantic elements, multimedia support, and APIs
- HTML files have the extension **.html** or **.htm**

1.2 HTML Document Structure

Every HTML document follows a standard structure. Understanding this structure is fundamental to creating valid web pages.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Web Page</title>
</head>
<body>
    <h1>Welcome to HTML</h1>
    <p>This is my first paragraph.</p>
</body>
</html>
```

Explanation of Each Part:

- **<!DOCTYPE html>** - Declares the document type as HTML5. Must be the first line.
- **<html>** - The root element that wraps all content. The 'lang' attribute specifies language.
- **<head>** - Contains meta-information about the document (not displayed on page).
- **<meta charset="UTF-8">** - Specifies character encoding for proper text display.
- **<meta name="viewport">** - Controls viewport behavior for responsive design.
- **<title>** - Defines the title shown in browser tab and search results.
- **<body>** - Contains all visible content of the web page.

1.3 HTML Elements & Tags

An HTML element typically consists of a start tag, content, and an end tag. Some elements are self-closing (void elements) and do not have content or end tags.

Structure of an Element:

```
<tagname attribute="value">Content goes here</tagname>

Example:
<p class="intro">This is a paragraph.</p>

Self-closing (void) elements:
<img src="image.jpg" alt="description">
<br>
<hr>
<input type="text">
```

Common HTML Tags:

Tag	Description	Example
<h1> to <h6>	Headings (h1 largest, h6 smallest)	`<h1>Main Title</h1>`
<p>	Paragraph	`<p>Some text</p>`
<a>	Anchor (hyperlink)	`<a href="url">Link</a>`
<img>	Image (void element)	`<img src="pic.jpg">`
** **	Block-level container	`<div>Content</div>`
****	Inline container	`<span>Text</span>`
** **	Line break (void)	`Line 1 Line 2`
<hr>	Horizontal rule (void)	`<hr>`
****	Important text (bold)	`<strong>Important</strong>`
****	Emphasized text (italic)	`<em>Emphasis</em>`
** , , **	Lists (unordered/ordered)	`<ul><li>Item</li></ul>`
<table>	Table	`<table>...</table>`
<form>	Form container	`<form>...</form>`
<input>	Input field (void)	`<input type="text">`

1.4 HTML Attributes

Attributes provide additional information about HTML elements. They are always specified in the start tag and usually come in name/value pairs like name="value".

Common Global Attributes:

- **id** - Unique identifier for an element (must be unique on page)
- **class** - One or more class names for CSS/JavaScript targeting
- **style** - Inline CSS styling

- **title** - Extra information shown as tooltip on hover
- **lang** - Language of element's content
- **data-*** - Custom data attributes for storing extra information
- **hidden** - Hides the element from display
- **tabindex** - Controls tab order for keyboard navigation

```
<!-- Examples of attributes -->
<div id="header" class="main-header" style="color: blue;">
  <p title="Hover for info">Welcome</p>
</div>

<img src="logo.png" alt="Company Logo" width="200" height="100">

<a href="https://example.com" target="_blank" rel="noopener">
  Visit Site
</a>

<!-- Custom data attribute -->
<button data-user-id="123" data-action="delete">Delete</button>
```

1.5 Text Formatting

HTML provides several elements for formatting text. It's important to use semantic tags that convey meaning, not just visual styling.

Tag	Purpose	Visual Effect
<b>	Bold text (non-semantic)	Bold
****	Important text (semantic)	Bold
<i>	Italic text (non-semantic)	Italic
****	Emphasized text (semantic)	Italic
<u>	Underlined text	Underlined
<mark>	Highlighted text	Yellow background
<small>	Smaller text	Smaller font
<del>	Deleted text	Strikethrough
<ins>	Inserted text	Underlined
<sub>	Subscript	H■O effect
<sup>	Superscript	x² effect
<code>	Inline code	Monospace font
<pre>	Preformatted text	Preserves whitespace
<blockquote>	Block quotation	Indented

1.6 Lists

HTML supports three types of lists: unordered lists, ordered lists, and description lists.

Unordered List (Bullets):

```
<ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ul>
```

Ordered List (Numbered):

```
<ol type="1" start="1">
    <li>First step</li>
    <li>Second step</li>
    <li>Third step</li>
</ol>

<!-- Types: 1, A, a, I, i -->
<ol type="A">  <!-- A, B, C... -->
    <li>Item A</li>
</ol>
```

Description List:

```
<dl>
    <dt>HTML</dt>
    <dd>Markup language for web pages</dd>
    <dt>CSS</dt>
    <dd>Style sheet language</dd>
</dl>
```

Nested Lists:

```
<ul>
    <li>Frontend
        <ul>
            <li>HTML</li>
            <li>CSS</li>
            <li>JavaScript</li>
        </ul>
    </li>
    <li>Backend
        <ul>
            <li>Node.js</li>
            <li>Python</li>
        </ul>
    </li>
</ul>
```

1.7 Links & Anchors

Hyperlinks are created using the <a> (anchor) tag. They allow users to navigate between pages, sections, or external resources.

```
<!-- Basic link -->
<a href="https://example.com">Visit Example</a>

<!-- Link opens in new tab -->
<a href="https://example.com" target="_blank" rel="noopener noreferrer">
    Open in New Tab
</a>
```

```
<!-- Link to email -->
<a href="mailto:hello@example.com">Send Email</a>

<!-- Link to phone -->
<a href="tel:+1234567890">Call Us</a>

<!-- Link to specific section (anchor) -->
<a href="#section1">Go to Section 1</a>
<div id="section1">Section 1 content</div>

<!-- Relative vs Absolute paths -->
<a href="/about">About (absolute path)</a>
<a href="about.html">About (relative path)</a>
<a href="../index.html">Parent directory</a>
```

Important Attributes of <a>:

- **href** - The URL of the link destination
- **target** - Where to open the link: _self (default), _blank (new tab), _parent, _top
- **rel** - Relationship: noopener, noreferrer, nofollow (security/SEO)
- **download** - Prompts download instead of navigation
- **title** - Tooltip text on hover

1.8 Images

Images are embedded using the <img> tag. It is a void (self-closing) element and supports various formats like JPG, PNG, GIF, SVG, and WebP.

```
<!-- Basic image -->
<img src="photo.jpg" alt="A beautiful sunset">

<!-- Image with dimensions -->
<img src="logo.png" alt="Logo" width="200" height="100">

<!-- Responsive image with srcset -->
<img src="small.jpg"
     srcset="small.jpg 480w, medium.jpg 768w, large.jpg 1200w"
     sizes="(max-width: 600px) 480px, 100vw"
     alt="Responsive image">

<!-- Picture element for art direction -->
<picture>
    <source media="(min-width: 768px)" srcset="desktop.jpg">
    <source media="(min-width: 480px)" srcset="tablet.jpg">
    <img src="mobile.jpg" alt="Responsive picture">
</picture>

<!-- Image as link -->
<a href="page.html">
    <img src="button.png" alt="Click me">
</a>
```

Important Notes:

- **alt attribute is mandatory** - For accessibility (screen readers) and SEO
- Always specify width and height to prevent layout shift (CLS)
- Use appropriate image format: JPG for photos, PNG for transparency, SVG for icons, WebP for performance

- Use srcset and picture for responsive images
- Lazy load images with loading="lazy" attribute for performance

1.9 Tables

Tables are used to display tabular data in rows and columns. They should NOT be used for page layout (use CSS instead).

```
<table border="1">
    <caption>Employee Data</caption>
    <thead>
        <tr>
            <th>Name</th>
            <th>Role</th>
            <th>Salary</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>John Doe</td>
            <td>Developer</td>
            <td>$50,000</td>
        </tr>
        <tr>
            <td>Jane Smith</td>
            <td>Designer</td>
            <td>$45,000</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <td colspan="2">Total</td>
            <td>$95,000</td>
        </tr>
    </tfoot>
</table>
```

Table Elements:

- **<table>** - Main table container
- **<caption>** - Table title/caption
- **<thead>** - Group of header rows
- **<tbody>** - Group of body rows
- **<tfoot>** - Group of footer rows
- **<tr>** - Table row
- **<th>** - Header cell (bold, centered by default)
- **<td>** - Data cell
- **colspan** - Span multiple columns
- **rowspan** - Span multiple rows

1.10 Forms & Input Elements

Forms are used to collect user input. They are critical for any interactive web application and are heavily tested in interviews.

```
<form action="/submit" method="POST" enctype="multipart/form-data">
    <!-- Text input -->
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required
           placeholder="Enter your name" maxlength="50">

    <!-- Email input -->
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <!-- Password -->
    <label for="pwd">Password:</label>
    <input type="password" id="pwd" name="password" minlength="8">

    <!-- Number -->
    <input type="number" name="age" min="18" max="100" step="1">

    <!-- Radio buttons -->
    <input type="radio" id="male" name="gender" value="male">
    <label for="male">Male</label>
    <input type="radio" id="female" name="gender" value="female">
    <label for="female">Female</label>

    <!-- Checkboxes -->
    <input type="checkbox" id="terms" name="terms" required>
    <label for="terms">I agree to terms</label>

    <!-- Dropdown -->
    <label for="country">Country:</label>
    <select id="country" name="country">
        <option value="">--Select--</option>
        <option value="in">India</option>
        <option value="us">USA</option>
    </select>

    <!-- Textarea -->
    <label for="msg">Message:</label>
    <textarea id="msg" name="message" rows="4" cols="30"></textarea>

    <!-- File upload -->
    <input type="file" name="resume" accept=".pdf,.doc">

    <!-- Date -->
    <input type="date" name="birthday">

    <!-- Hidden field -->
    <input type="hidden" name="user_id" value="123">

    <!-- Submit and Reset -->
    <button type="submit">Submit</button>
    <button type="reset">Reset</button>
</form>
```

Common Input Types:

Type	Purpose
`text`	Single-line text input
`password`	Password input (masked)
`email`	Email with validation

`number`	Numeric input
`tel`	Telephone number
`url`	URL input
`date`	Date picker
`time`	Time picker
`datetime-local`	Date and time
`radio`	Single choice from group
`checkbox`	Multiple choices
`file`	File upload
`hidden`	Hidden data
`color`	Color picker
`range`	Slider input
`search`	Search field
`submit`	Submit button
`reset`	Reset button
`button`	Generic button

Form Attributes:

- **action** - URL where form data is submitted
- **method** - HTTP method: GET (data in URL) or POST (in body)
- **enctype** - Encoding type (use multipart/form-data for file uploads)
- **autocomplete** - Enable/disable browser autocomplete
- **novalidate** - Disable HTML5 validation
- **target** - Where to display response (_self, _blank, etc.)

Input Validation Attributes:

- **required** - Field must be filled
- **minlength / maxlength** - Min/max characters
- **min / max** - Min/max values for number/date
- **pattern** - Regex pattern to match
- **placeholder** - Hint text shown when empty
- **readonly** - User cannot edit
- **disabled** - Cannot be interacted with, not submitted
- **autofocus** - Field auto-focused on page load

1.11 Semantic HTML5

Semantic HTML uses tags that convey the meaning of content rather than just presentation. HTML5 introduced many semantic elements that improve accessibility, SEO, and code readability.

Why Semantic HTML Matters:

- **Accessibility** - Screen readers can better navigate content
- **SEO** - Search engines understand page structure better
- **Maintainability** - Code is more readable and self-documenting
- **Consistency** - Standard structure across web pages

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Semantic Layout</title>
</head>
<body>
    <header>
        <h1>My Website</h1>
        <nav>
            <ul>
                <li><a href="/">Home</a></li>
                <li><a href="/about">About</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <article>
            <header>
                <h2>Article Title</h2>
                <time datetime="2026-01-15">January 15, 2026</time>
            </header>
            <section>
                <h3>Introduction</h3>
                <p>Article introduction...</p>
            </section>
            <section>
                <h3>Main Content</h3>
                <p>Main content here...</p>
            </section>
        </article>

        <aside>
            <h3>Related Articles</h3>
            <ul>
                <li><a href="#">Article 1</a></li>
            </ul>
        </aside>
    </main>

    <footer>
        <p>&copy; 2026 My Website</p>
    </footer>
</body>
</html>
```

Element	Purpose
<header>	Introductory content or navigation
<nav>	Navigation links
<main>	Main content of the page (only one per page)
<article>	Self-contained content (blog post, news article)
<section>	Thematic grouping of content

<aside>	Sidebar content related to main content
<footer>	Footer content (copyright, contact info)
<figure>	Self-contained content like images, diagrams
<figcaption>	Caption for a figure element
<time>	Date/time information
<address>	Contact information
<details>	Disclosure widget (collapsible content)
<summary>	Summary/heading for <details>
<mark>	Highlighted/marked text

1.12 Multimedia (Audio & Video)

HTML5 introduced native support for audio and video, eliminating the need for plugins like Flash.

Video Element:

```
<video width="640" height="360" controls autoplay muted loop poster="thumb.jpg">
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    Your browser does not support the video tag.
</video>
```

Audio Element:

```
<audio controls>
    <source src="audio.mp3" type="audio/mpeg">
    <source src="audio.ogg" type="audio/ogg">
    Your browser does not support audio.
</audio>
```

Common Attributes:

- **controls** - Shows play/pause/volume controls
- **autoplay** - Starts playing automatically (often requires muted)
- **muted** - Audio muted by default
- **loop** - Replays when finished
- **preload** - none, metadata, or auto
- **poster** - Image shown before video plays (video only)

1.13 Meta Tags & SEO

Meta tags provide metadata about the HTML document. They are crucial for SEO, social sharing, and browser behavior.

```
<head>
    <!-- Character encoding -->
    <meta charset="UTF-8">

    <!-- Viewport for responsive design -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <!-- SEO meta tags -->
    <title>Page Title - Brand Name (50-60 chars)</title>
    <meta name="description" content="Page description for SEO (150-160 chars)">
    <meta name="keywords" content="html, css, bootstrap, web development">
    <meta name="author" content="Your Name">

    <!-- Robots -->
    <meta name="robots" content="index, follow">

    <!-- Open Graph (Facebook, LinkedIn) -->
    <meta property="og:title" content="Page Title">
    <meta property="og:description" content="Page description">
    <meta property="og:image" content="https://example.com/image.jpg">
    <meta property="og:url" content="https://example.com">
    <meta property="og:type" content="website">

    <!-- Twitter Card -->
    <meta name="twitter:card" content="summary_large_image">
    <meta name="twitter:title" content="Page Title">
    <meta name="twitter:description" content="Description">
    <meta name="twitter:image" content="https://example.com/image.jpg">

    <!-- Favicon -->
    <link rel="icon" type="image/png" href="favicon.png">

    <!-- Canonical URL (avoid duplicate content) -->
    <link rel="canonical" href="https://example.com/page">

    <!-- CSS link -->
    <link rel="stylesheet" href="styles.css">
</head>
```

1.14 Accessibility (a11y)

Web accessibility ensures that websites are usable by people with disabilities. It's a critical interview topic for modern web development.

Best Practices:

- Use **semantic HTML** elements (header, nav, main, etc.)
- Always include **alt** attribute on images
- Use **proper heading hierarchy** (h1 → h2 → h3, don't skip)
- Associate **labels** with form controls using 'for' attribute
- Ensure sufficient **color contrast** (WCAG AA: 4.5:1 for normal text)
- Make site **keyboard navigable** (Tab order, focus indicators)
- Use **ARIA attributes** when needed (aria-label, aria-hidden, role)
- Provide **skip navigation links** for screen readers
- Use **captions** for videos and transcripts for audio

```
<!-- Accessibility examples -->

<!-- Proper image alt text -->
<img src="dog.jpg" alt="Golden retriever playing in park">

<!-- Decorative image (empty alt) -->
<img src="decoration.png" alt="" role="presentation">

<!-- Form with proper labels -->
```

```
<label for="email">Email Address</label>
<input type="email" id="email" name="email"
       aria-describedby="email-help" required>
<span id="email-help">We'll never share your email</span>

<!-- ARIA for custom components -->
<button aria-label="Close modal" aria-expanded="false">
    <span aria-hidden="true">&times;</span>
</button>

<!-- Skip navigation -->
<a href="#main-content" class="skip-link">Skip to main content</a>

<!-- Live region for dynamic updates -->
<div aria-live="polite" aria-atomic="true" id="status">
    Loading...
</div>
```

PART 2: CSS MASTERY

2.1 Introduction to CSS

CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation of HTML documents. It controls layout, colors, fonts, animations, and responsive behavior.

Three Ways to Apply CSS:

```
<!-- 1. Inline CSS (highest specificity, avoid except for dynamic styles) -->
<p style="color: red; font-size: 18px;">Inline styled text</p>

<!-- 2. Internal CSS (in <head> section) -->
<head>
    <style>
        p { color: blue; }
        .highlight { background: yellow; }
    </style>
</head>

<!-- 3. External CSS (best practice) -->
<head>
    <link rel="stylesheet" href="styles.css">
</head>
```

Why Use External CSS:

- **Separation of concerns** - HTML for structure, CSS for style
- **Reusability** - One stylesheet for multiple pages
- **Maintainability** - Change styles in one place
- **Performance** - Browsers can cache external CSS files
- **Cleaner HTML** - Easier to read and edit

2.2 CSS Syntax & Selectors

CSS rules consist of selectors and declaration blocks. The selector targets HTML elements, and the declaration block contains property-value pairs.

```
/* CSS Syntax */
selector {
    property: value;
    property: value;
}

/* Example */
h1 {
    color: blue;
    font-size: 32px;
    margin-bottom: 20px;
}
```

Types of Selectors:

Selector	Example	Description
Universal	`*`	Selects all elements
Element/Type	`p, h1`	Selects all elements of that type
Class	`.class-name`	Selects elements with the class
ID	`#id-name`	Selects element with the ID (unique)
Attribute	`[type="text"]`	Selects by attribute value
Descendant	`div p`	p inside div (any level)
Child	`div > p`	p that is direct child of div
Adjacent sibling	`h1 + p`	p directly after h1
General sibling	`h1 ~ p`	All p siblings after h1
Pseudo-class	`a:hover`	a when hovered
Pseudo-element	`p::first-letter`	First letter of p
Grouping	`h1, h2, h3`	Multiple selectors

Pseudo-Classes:

```
/* User action */
a:hover { color: red; }       /* When hovered */
a:focus { outline: 2px solid blue; }  /* When focused */
a:active { color: orange; }   /* While being clicked */
a:visited { color: purple; }  /* Visited links */

/* Structural */
li:first-child { font-weight: bold; }
li:last-child { color: red; }
li:nth-child(2) { background: yellow; }      /* 2nd child */
li:nth-child(odd) { background: #eee; }      /* odd children */
li:nth-child(even) { background: #fff; }     /* even children */
li:nth-child(3n+1) { color: blue; }          /* every 3rd starting from 1 */

/* Form states */
input:focus { border-color: blue; }
input:disabled { opacity: 0.5; }
input:checked + label { color: green; }
input:required { border-left: 3px solid red; }
input:valid { border-color: green; }
input:invalid { border-color: red; }

/* Negation */
li:not(.special) { color: gray; }

/* Empty */
p:empty { display: none; }
```

Pseudo-Elements:

```
/* Content additions */
p::before { content: ">> "; color: blue; }
p::after { content: " <<"; color: blue; }

/* Text styling */
p::first-letter { font-size: 2em; color: red; }
p::first-line { font-weight: bold; }
```

```
/* Selection */
::selection { background: yellow; color: black; }

/* Placeholder */
input::placeholder { color: gray; font-style: italic; }
```

2.3 Specificity & Cascade

When multiple CSS rules apply to the same element, the browser uses specificity, source order, and importance to determine which rule wins. This is a VERY common interview topic.

Specificity Hierarchy (highest to lowest):

- **!important** - Overrides everything (use sparingly!)
- **Inline styles** - 1000 points (style="...")
- **IDs** - 100 points (#header)
- **Classes, attributes, pseudo-classes** - 10 points (.btn, [type], :hover)
- **Elements, pseudo-elements** - 1 point (div, ::before)
- **Universal selector (*)** - 0 points

```
/* Specificity calculation examples */

* { color: black; }              /* 0,0,0,0 - 0 */
p { color: blue; }               /* 0,0,0,1 - 1 */
.text { color: green; }          /* 0,0,1,0 - 10 */
p.text { color: orange; }        /* 0,0,1,1 - 11 */
#main { color: red; }            /* 0,1,0,0 - 100 */
#main p.text { color: pink; }    /* 0,1,1,1 - 111 */
<p style="color: gray">          /* 1,0,0,0 - 1000 */
p { color: navy !important; }    /* Highest - overrides all */

/* When same specificity, last one wins (source order) */
.btn { color: blue; }
.btn { color: red; }  /* This wins */
```

The Cascade (CSS rule priority):

- **1. Importance** - !important rules first
- **2. Specificity** - More specific selectors win
- **3. Source order** - Later rules override earlier ones
- **4. Inheritance** - Some properties inherit from parent (color, font), some don't (margin, border)

2.4 The Box Model

Every HTML element is a box. The CSS Box Model describes how the size of these boxes is calculated. This is one of the MOST asked interview topics.

Box Model Components (from inside out):

- **Content** - The actual content (text, image)
- **Padding** - Space between content and border (inside)
- **Border** - Border around padding
- **Margin** - Space outside border (between elements)

```
.box {
    /* Content size */
    width: 200px;
    height: 100px;

    /* Padding (all sides) */
    padding: 20px;
    /* Or specific sides */
    padding-top: 10px;
    padding-right: 15px;
    padding-bottom: 10px;
    padding-left: 15px;
    /* Shorthand: top right bottom left */
    padding: 10px 15px 10px 15px;
    /* Shorthand: vertical horizontal */
    padding: 10px 15px;

    /* Border */
    border: 2px solid black;
    /* Or specific */
    border-width: 2px;
    border-style: solid;  /* solid, dashed, dotted, double */
    border-color: black;
    border-radius: 10px;  /* Rounded corners */

    /* Margin */
    margin: 20px auto;  /* auto centers horizontally */
}
```

box-sizing Property (Critical!):

```
/* Default behavior */
.box {
    box-sizing: content-box;  /* DEFAULT */
    width: 200px;     /* Just content */
    padding: 20px;    /* Adds 40px (20 each side) */
    border: 2px;      /* Adds 4px (2 each side) */
    /* TOTAL WIDTH = 200 + 40 + 4 = 244px */
}

/* Better behavior (most developers prefer) */
.box {
    box-sizing: border-box;
    width: 200px;     /* INCLUDES padding & border */
    padding: 20px;    /* Subtracted from width */
    border: 2px;
    /* TOTAL WIDTH = 200px (exact!) */
}

/* Common reset */
*, *::before, *::after {
    box-sizing: border-box;
}
```

Margin Collapsing: When two vertical margins meet, they collapse to the larger of the two. This only happens with vertical margins, not horizontal. Margins don't collapse if there's padding, border, or content between them.

2.5 Display & Positioning

Display Property:

Value	Behavior
`block`	Takes full width, starts on new line (div, p, h1)
`inline`	Takes only needed width, no line break (span, a)
`inline-block`	Inline but accepts width/height
`none`	Hidden, removed from layout
`flex`	Flexbox container
`grid`	Grid container
`table`	Behaves like a table

Visibility vs Display:

- **display: none** - Removes element from layout (takes no space)
- **visibility: hidden** - Hides element but reserves space
- **opacity: 0** - Invisible but still clickable, takes space

Position Property:

```
/* static - DEFAULT, normal flow */
.element { position: static; }

/* relative - relative to its NORMAL position */
.element {
    position: relative;
    top: 10px;
    left: 20px;  /* Moves 20px right from normal position */
}

/* absolute - relative to nearest POSITIONED ancestor */
.parent { position: relative; }  /* Reference point */
.element {
    position: absolute;
    top: 0;
    right: 0;  /* Top-right of parent */
}

/* fixed - relative to viewport (stays on scroll) */
.header {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    z-index: 1000;
}

/* sticky - hybrid: relative until scroll threshold, then fixed */
.element {
    position: sticky;
    top: 0;  /* Sticks at top of viewport */
}
```

z-index Property:

- Controls stacking order of **positioned elements** (not static)
- Higher values appear on top
- Default is 'auto' (0)

- Can be negative (-1 puts behind)
- Creates new 'stacking context' for children

2.6 Flexbox (Flexible Box Layout)

Flexbox is a one-dimensional layout system for arranging items in rows or columns. It's extremely powerful for building responsive layouts and is heavily tested in interviews.

Flexbox Concept:

- Flex container has **main axis** and **cross axis**
- When flex-direction is 'row': main axis is horizontal, cross axis is vertical
- When flex-direction is 'column': main axis is vertical, cross axis is horizontal
- Items inside become 'flex items'

Container Properties:

```
.container {
    display: flex;  /* or inline-flex */

    /* Direction of main axis */
    flex-direction: row;          /* DEFAULT - left to right */
    flex-direction: row-reverse;  /* right to left */
    flex-direction: column;       /* top to bottom */
    flex-direction: column-reverse;

    /* Wrapping */
    flex-wrap: nowrap;    /* DEFAULT - single line */
    flex-wrap: wrap;      /* multiple lines */
    flex-wrap: wrap-reverse;

    /* Shorthand: flex-direction + flex-wrap */
    flex-flow: row wrap;

    /* Alignment along MAIN axis */
    justify-content: flex-start;    /* DEFAULT */
    justify-content: flex-end;
    justify-content: center;
    justify-content: space-between; /* equal space between */
    justify-content: space-around;  /* equal space around */
    justify-content: space-evenly;  /* truly equal spacing */

    /* Alignment along CROSS axis (single line) */
    align-items: stretch;     /* DEFAULT - fills container */
    align-items: flex-start;
    align-items: flex-end;
    align-items: center;
    align-items: baseline;

    /* Alignment of multiple lines on cross axis */
    align-content: flex-start;
    align-content: center;
    align-content: space-between;
    align-content: stretch;

    /* Gap between items */
    gap: 10px;
    row-gap: 10px;
    column-gap: 20px;
}
```

Item Properties:

```
.item {
    /* Order (default 0, lower comes first) */
    order: 1;

    /* How much it grows relative to others */
    flex-grow: 1;  /* 0 = don't grow */

    /* How much it shrinks */
    flex-shrink: 1;  /* 0 = don't shrink */

    /* Initial size before flex */
    flex-basis: 200px;  /* or auto, % */

    /* SHORTHAND: grow shrink basis */
    flex: 1 1 200px;
    flex: 1;        /* = 1 1 0% */
    flex: auto;     /* = 1 1 auto */
    flex: none;     /* = 0 0 auto */

    /* Override align-items for individual item */
    align-self: center;
}
```

Common Flexbox Patterns:

```
/* Center anything (horizontally and vertically) */
.center {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

/* Equal width columns */
.columns {
    display: flex;
}
.columns > div {
    flex: 1;
}

/* Header with logo + nav (space between) */
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

/* Card with footer pushed to bottom */
.card {
    display: flex;
    flex-direction: column;
    min-height: 300px;
}
.card-footer {
    margin-top: auto;  /* Pushes to bottom */
}

/* Responsive navbar */
.nav {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
```

```
}
```

2.7 CSS Grid

CSS Grid is a two-dimensional layout system, perfect for creating complex grid-based layouts. While Flexbox is 1D, Grid handles both rows AND columns simultaneously.

Container Properties:

```
.grid {
    display: grid;

    /* Define columns */
    grid-template-columns: 200px 200px 200px;    /* 3 fixed columns */
    grid-template-columns: 1fr 1fr 1fr;          /* 3 equal columns */
    grid-template-columns: 200px 1fr 1fr;        /* fixed + flexible */
    grid-template-columns: repeat(3, 1fr);       /* 3 equal columns */
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  /* Responsive! */

    /* Define rows */
    grid-template-rows: 100px 200px auto;
    grid-template-rows: repeat(3, 100px);

    /* Gap between cells */
    gap: 20px;              /* row and column gap */
    row-gap: 10px;
    column-gap: 20px;

    /* Alignment of items WITHIN cells */
    justify-items: stretch;  /* horizontal: start, end, center, stretch */
    align-items: stretch;    /* vertical */

    /* Alignment of GRID within container */
    justify-content: start;
    align-content: start;

    /* Named grid areas */
    grid-template-areas:
        "header header header"
        "sidebar main main"
        "footer footer footer";
}
```

Item Properties:

```
.item {
    /* Place item across columns */
    grid-column-start: 1;
    grid-column-end: 3;
    /* Shorthand */
    grid-column: 1 / 3;        /* From line 1 to line 3 */
    grid-column: 1 / span 2;   /* Span 2 columns */

    /* Place across rows */
    grid-row: 1 / 3;

    /* Place in named area */
    grid-area: header;

    /* Self alignment */
    justify-self: center;
    align-self: center;
}
```

Complete Grid Layout Example:

```
.layout {
    display: grid;
    grid-template-columns: 200px 1fr;
    grid-template-rows: 80px 1fr 60px;
    grid-template-areas:
        "header  header"
        "sidebar main"
        "footer  footer";
    min-height: 100vh;
    gap: 10px;
}

.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }
```

Flexbox vs Grid - When to Use:

Use Flexbox	Use Grid
One dimension (row OR column)	Two dimensions (rows AND columns)
Navigation bars	Page layouts
Card components	Image galleries
Centering items	Dashboard layouts
Form layouts	Magazine-style layouts
Content-first design	Layout-first design

2.8 Responsive Design & Media Queries

Responsive design ensures your website looks great on all devices - phones, tablets, laptops, and desktops. Media queries are the foundation of responsive design.

Viewport Meta Tag (REQUIRED):

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Media Queries:

```
/* Mobile-first approach (RECOMMENDED) */
.container {
    width: 100%;
    padding: 10px;
}

/* Tablet (768px and up) */
@media (min-width: 768px) {
    .container {
        max-width: 720px;
        margin: 0 auto;
        padding: 20px;
    }
}

/* Desktop (1024px and up) */
```

```
@media (min-width: 1024px) {
    .container {
        max-width: 960px;
        padding: 30px;
    }
}

/* Large desktop (1200px and up) */
@media (min-width: 1200px) {
    .container {
        max-width: 1140px;
    }
}

/* Targeting specific ranges */
@media (min-width: 768px) and (max-width: 1023px) {
    /* Tablet only */
}

/* Orientation */
@media (orientation: landscape) {
    /* Landscape mode */
}

@media (orientation: portrait) {
    /* Portrait mode */
}

/* Print styles */
@media print {
    nav, footer { display: none; }
    body { color: black; background: white; }
}

/* Hover support (touch devices typically don't support hover) */
@media (hover: hover) {
    .btn:hover { background: blue; }
}

/* Dark mode */
@media (prefers-color-scheme: dark) {
    body { background: #1a1a1a; color: #fff; }
}
```

Common Breakpoints:

Device	Breakpoint
Mobile (small)	< 576px
Mobile (large)	576px - 767px
Tablet	768px - 991px
Desktop	992px - 1199px
Large Desktop	≥ 1200px
Extra Large	≥ 1400px

2.9 Units, Colors & Typography

CSS Units:

Unit	Type	Description
px	Absolute	Pixels (fixed size)
pt	Absolute	Points (1pt = 1/72 inch) - print
cm, mm, in	Absolute	Physical units (for print)
%	Relative	Percent of parent
em	Relative	Relative to PARENT font-size
rem	Relative	Relative to ROOT font-size (best!)
vw	Relative	1% of viewport width
vh	Relative	1% of viewport height
vmin	Relative	1% of smaller dimension
vmax	Relative	1% of larger dimension
ch	Relative	Width of "0" character
ex	Relative	Height of "x" character
fr	Grid	Fraction of available space (grid)

Colors:

```
/* Named colors */
color: red;
color: blueviolet;

/* HEX */
color: #ff0000;       /* Red */
color: #f00;          /* Short HEX (same as #ff0000) */
color: #ff000080;     /* HEX with alpha (50% transparent) */

/* RGB / RGBA */
color: rgb(255, 0, 0);
color: rgba(255, 0, 0, 0.5);  /* 50% transparent */

/* HSL / HSLA */
color: hsl(0, 100%, 50%);      /* Red */
color: hsla(0, 100%, 50%, 0.5);

/* Modern syntax */
color: rgb(255 0 0 / 0.5);
color: hsl(0 100% 50% / 0.5);
```

Typography:

```
.text {
    /* Font family with fallbacks */
    font-family: 'Roboto', Arial, sans-serif;

    /* Font size */
    font-size: 16px;        /* or 1rem */

    /* Font weight */
    font-weight: 400;       /* normal */
    font-weight: 700;       /* bold */
    font-weight: 100-900;
```

```
    /* Font style */
    font-style: italic;     /* normal, italic, oblique */

    /* Line height (vertical space) */
    line-height: 1.5;       /* 1.5x font size */
    line-height: 24px;

    /* Letter spacing */
    letter-spacing: 0.05em;

    /* Word spacing */
    word-spacing: 0.1em;

    /* Text alignment */
    text-align: left;       /* left, right, center, justify */

    /* Text decoration */
    text-decoration: underline;     /* underline, overline, line-through */
    text-decoration: none;

    /* Text transform */
    text-transform: uppercase;      /* lowercase, capitalize */

    /* Text indent (first line) */
    text-indent: 30px;

    /* Shadow */
    text-shadow: 2px 2px 4px rgba(0,0,0,0.5);

    /* Shorthand */
    font: italic bold 16px/1.5 Arial, sans-serif;
    /* style weight size/line-height family */
}

/* Web fonts */
@font-face {
    font-family: 'MyFont';
    src: url('myfont.woff2') format('woff2'),
         url('myfont.woff') format('woff');
    font-display: swap;
}
```

2.10 Transitions & Animations

CSS Transitions:

Transitions allow you to smoothly change property values over a specified duration. Perfect for hover effects and state changes.

```
.button {
    background: blue;
    color: white;
    transform: scale(1);

    /* Transition syntax: property duration timing-function delay */
    transition: background 0.3s ease, transform 0.2s ease-in-out;

    /* Or transition all properties */
    transition: all 0.3s ease;
}

.button:hover {
    background: darkblue;
```

```
    transform: scale(1.05);
}

/* Timing functions */
transition-timing-function: linear;
transition-timing-function: ease;       /* DEFAULT */
transition-timing-function: ease-in;
transition-timing-function: ease-out;
transition-timing-function: ease-in-out;
transition-timing-function: cubic-bezier(0.25, 0.1, 0.25, 1.0);
```

CSS Animations (Keyframes):

Animations give you full control over a sequence of changes using keyframes.

```
/* Define keyframes */
@keyframes slideIn {
    from {
        opacity: 0;
        transform: translateX(-100%);
    }
    to {
        opacity: 1;
        transform: translateX(0);
    }
}

/* Multiple keyframes */
@keyframes bounce {
    0%   { transform: translateY(0); }
    50%  { transform: translateY(-50px); }
    100% { transform: translateY(0); }
}

@keyframes loading {
    0%   { background: red; }
    25%  { background: yellow; }
    50%  { background: green; }
    75%  { background: blue; }
    100% { background: red; }
}

/* Apply animation */
.element {
    animation: slideIn 1s ease-in-out;

    /* Or detailed */
    animation-name: slideIn;
    animation-duration: 1s;
    animation-timing-function: ease-in-out;
    animation-delay: 0.5s;
    animation-iteration-count: 3;        /* or 'infinite' */
    animation-direction: alternate;       /* normal, reverse, alternate */
    animation-fill-mode: forwards;        /* none, forwards, backwards, both */
    animation-play-state: running;        /* running, paused */
}

/* Common loading spinner */
@keyframes spin {
    from { transform: rotate(0deg); }
    to   { transform: rotate(360deg); }
}

.spinner {
    width: 40px; height: 40px;
```

```
    border: 4px solid #f3f3f3;
    border-top: 4px solid #3498db;
    border-radius: 50%;
    animation: spin 1s linear infinite;
}
```

2.11 Transforms

CSS transforms let you rotate, scale, skew, and translate elements without affecting surrounding elements (they don't trigger layout reflow - great for performance).

```
.element {
    /* 2D Transforms */
    transform: translate(50px, 100px);    /* Move x, y */
    transform: translateX(50px);
    transform: translateY(100px);

    transform: scale(1.5);                /* Scale uniformly */
    transform: scale(2, 0.5);             /* scale-x, scale-y */
    transform: scaleX(2);
    transform: scaleY(0.5);

    transform: rotate(45deg);             /* Rotate */
    transform: rotate(-90deg);

    transform: skew(20deg, 10deg);        /* Skew */
    transform: skewX(20deg);
    transform: skewY(10deg);

    /* Combine multiple transforms */
    transform: translate(50px, 100px) rotate(45deg) scale(1.5);

    /* Set origin point (default is center) */
    transform-origin: top left;
    transform-origin: 50% 50%;
    transform-origin: 100px 200px;

    /* 3D Transforms */
    transform: translate3d(50px, 100px, 50px);
    transform: rotateX(45deg);
    transform: rotateY(45deg);
    transform: rotateZ(45deg);
    transform: perspective(500px) rotateY(45deg);
}
```

2.12 CSS Variables (Custom Properties)

CSS Variables allow you to define reusable values. They are crucial for theming, maintainability, and dynamic styling.

```
/* Define variables in :root (global) */
:root {
    --primary-color: #1565c0;
    --secondary-color: #f44336;
    --text-color: #212121;
    --font-base: 16px;
    --spacing: 1rem;
    --border-radius: 8px;
    --max-width: 1200px;
}
```

```
/* Use variables */
.button {
    background: var(--primary-color);
    color: white;
    padding: var(--spacing);
    border-radius: var(--border-radius);
    font-size: var(--font-base);
}

/* Fallback value (if variable undefined) */
.element {
    color: var(--text-color, black);
}

/* Override in scope */
.dark-theme {
    --primary-color: #90caf9;
    --text-color: #fff;
    --bg-color: #1a1a1a;
}

/* Change with JavaScript */
/* document.documentElement.style.setProperty('--primary-color', 'red'); */
```

Advantages of CSS Variables: Can be updated dynamically with JavaScript, respect the cascade and inheritance, work in media queries, and are perfect for implementing dark/light themes.

Bonus: CSS Methodologies

Industry-standard approaches to writing maintainable CSS:

- **BEM (Block Element Modifier)** - Naming convention: .block__element--modifier
- **SMACSS** - Scalable and Modular Architecture for CSS
- **OOCSS** - Object-Oriented CSS (separation of structure and skin)
- **Atomic CSS / Utility-first** - Small, single-purpose classes (Tailwind approach)
- **CSS-in-JS** - Styled-components, Emotion (React ecosystem)

```
/* BEM Example */
.card { }                    /* Block */
.card__title { }             /* Element */
.card__title--large { }      /* Modifier */
.card--featured { }          /* Block modifier */

/* HTML */
<div class="card card--featured">
    <h2 class="card__title card__title--large">Title</h2>
    <p class="card__text">Description</p>
</div>
```

PART 3: BOOTSTRAP FRAMEWORK

3.1 Introduction to Bootstrap

Bootstrap is the world's most popular free and open-source CSS framework for building responsive, mobile-first websites. Originally created by Twitter, it provides pre-built components, utilities, and a powerful grid system.

Why Use Bootstrap?

- **Mobile-first responsive design** built-in
- **Cross-browser compatibility** - works consistently across browsers
- **Rich component library** - buttons, modals, navbars, forms, etc.
- **Powerful grid system** based on Flexbox (12-column)
- **Customizable** via Sass variables
- **Large community** and extensive documentation
- **Saves time** - rapid prototyping and development
- **Latest version: Bootstrap 5** (no jQuery dependency)

Bootstrap 5 vs Bootstrap 4 (Key Differences):

Feature	Bootstrap 4	Bootstrap 5
jQuery	Required	Removed (vanilla JS)
IE support	Yes	Dropped (IE11 removed)
CSS variables	Limited	Extensive use
Grid	Has 5 breakpoints	Has 6 breakpoints (added xxl)
Forms	Custom checkboxes/radios	Redesigned form controls
Utilities	Limited	Utility API
Icons	Glyphicons (dropped)	Bootstrap Icons library
RTL support	Limited	Built-in RTL

3.2 Installation & Setup

Method 1: CDN (Quickest):

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Bootstrap 5 Demo</title>

    <!-- Bootstrap CSS -->
```

```
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
          rel="stylesheet">

    <!-- Bootstrap Icons (optional) -->
    <link rel="stylesheet"
          href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
</head>
<body>
    <h1 class="text-center">Hello Bootstrap!</h1>

    <!-- Bootstrap JS Bundle (with Popper) -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
    </script>
</body>
</html>
```

Method 2: NPM:

```
npm install bootstrap

// In your JS file
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap/dist/js/bootstrap.bundle.min.js';
```

3.3 Grid System

Bootstrap's grid system uses Flexbox and provides a 12-column responsive layout. Understanding the grid is FUNDAMENTAL for Bootstrap interviews.

Three Required Components:

- **.container** or **.container-fluid** - Wraps the grid
- **.row** - Horizontal group of columns
- **.col-*** - Individual columns (must be inside .row)

Breakpoints (Bootstrap 5):

Breakpoint	Class Prefix	Min Width	Description
Extra small	`.col-`	<576px	Phones (portrait)
Small	`.col-sm-`	≥576px	Phones (landscape)
Medium	`.col-md-`	≥768px	Tablets
Large	`.col-lg-`	≥992px	Desktops
Extra large	`.col-xl-`	≥1200px	Large desktops
XXL	`.col-xxl-`	≥1400px	Larger desktops

Containers:

```
<!-- Fixed-width container (responsive max-width) -->
<div class="container">...</div>

<!-- Full-width container -->
<div class="container-fluid">...</div>

<!-- Responsive containers (100% width until breakpoint) -->
```

```
<div class="container-sm">100% wide until small</div>
<div class="container-md">100% wide until medium</div>
<div class="container-lg">100% wide until large</div>
```

Basic Grid Examples:

```
<!-- Equal-width columns -->
<div class="container">
    <div class="row">
        <div class="col">Column 1</div>
        <div class="col">Column 2</div>
        <div class="col">Column 3</div>
    </div>
</div>

<!-- Specific column widths (total should = 12) -->
<div class="row">
    <div class="col-4">4/12 width</div>
    <div class="col-8">8/12 width</div>
</div>

<!-- Responsive columns -->
<div class="row">
    <div class="col-12 col-md-6 col-lg-4">
        Full width on mobile, half on tablet, third on desktop
    </div>
    <div class="col-12 col-md-6 col-lg-4">Same</div>
    <div class="col-12 col-md-12 col-lg-4">Same</div>
</div>

<!-- Auto-layout columns (one wider) -->
<div class="row">
    <div class="col">Equal</div>
    <div class="col-6">Twice as wide</div>
    <div class="col">Equal</div>
</div>

<!-- Offset (push column) -->
<div class="row">
    <div class="col-4 offset-4">Centered (4 wide, 4 offset)</div>
</div>

<!-- Reorder columns -->
<div class="row">
    <div class="col order-2">First in HTML, 2nd visually</div>
    <div class="col order-1">Second in HTML, 1st visually</div>
</div>

<!-- Nested grids -->
<div class="row">
    <div class="col-6">
        <div class="row">
            <div class="col-6">Nested 1</div>
            <div class="col-6">Nested 2</div>
        </div>
    </div>
</div>
```

Gutters (spacing between columns):

```
<!-- Default gutter -->
<div class="row">...</div>

<!-- No gutter -->
<div class="row g-0">...</div>
```

```
<!-- Custom gutter (0-5) -->
<div class="row g-3">...</div>     <!-- 1rem gutter -->
<div class="row gx-5">...</div>    <!-- Horizontal only -->
<div class="row gy-2">...</div>    <!-- Vertical only -->
```

3.4 Common Components

Buttons:

```
<!-- Color variants -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-info">Info</button>
<button class="btn btn-light">Light</button>
<button class="btn btn-dark">Dark</button>
<button class="btn btn-link">Link</button>

<!-- Outline -->
<button class="btn btn-outline-primary">Outline Primary</button>

<!-- Sizes -->
<button class="btn btn-primary btn-lg">Large</button>
<button class="btn btn-primary">Default</button>
<button class="btn btn-primary btn-sm">Small</button>

<!-- Block button (full width) -->
<button class="btn btn-primary w-100">Full Width</button>

<!-- Button group -->
<div class="btn-group">
    <button class="btn btn-primary">Left</button>
    <button class="btn btn-primary">Middle</button>
    <button class="btn btn-primary">Right</button>
</div>

<!-- Disabled -->
<button class="btn btn-primary" disabled>Disabled</button>
```

Navbar:

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
    <div class="container-fluid">
        <a class="navbar-brand" href="#">Logo</a>

        <button class="navbar-toggler" type="button"
                data-bs-toggle="collapse" data-bs-target="#navMenu">
            <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse" id="navMenu">
            <ul class="navbar-nav me-auto">
                <li class="nav-item">
                    <a class="nav-link active" href="#">Home</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="#">About</a>
                </li>
                <li class="nav-item dropdown">
                    <a class="nav-link dropdown-toggle" href="#"
                       data-bs-toggle="dropdown">Services</a>
```

```
                    <ul class="dropdown-menu">
                        <li><a class="dropdown-item" href="#">Service 1</a></li>
                        <li><a class="dropdown-item" href="#">Service 2</a></li>
                    </ul>
                </li>
            </ul>

            <form class="d-flex">
                <input class="form-control me-2" type="search" placeholder="Search">
                <button class="btn btn-outline-success">Search</button>
            </form>
        </div>
    </div>
</nav>
```

Cards:

```
<div class="card" style="width: 18rem;">
    <img src="image.jpg" class="card-img-top" alt="...">
    <div class="card-body">
        <h5 class="card-title">Card Title</h5>
        <h6 class="card-subtitle mb-2 text-muted">Subtitle</h6>
        <p class="card-text">Some quick example text.</p>
        <a href="#" class="card-link">Link</a>
        <a href="#" class="btn btn-primary">Button</a>
    </div>
    <div class="card-footer text-muted">2 days ago</div>
</div>

<!-- Card grid -->
<div class="row row-cols-1 row-cols-md-3 g-4">
    <div class="col">
        <div class="card h-100">...</div>
    </div>
    <!-- More cards -->
</div>
```

Forms:

```
<form>
    <div class="mb-3">
        <label for="email" class="form-label">Email</label>
        <input type="email" class="form-control" id="email"
               placeholder="name@example.com">
        <div class="form-text">We'll never share your email.</div>
    </div>

    <div class="mb-3">
        <label for="password" class="form-label">Password</label>
        <input type="password" class="form-control" id="password">
    </div>

    <div class="mb-3 form-check">
        <input type="checkbox" class="form-check-input" id="remember">
        <label class="form-check-label" for="remember">Remember me</label>
    </div>

    <select class="form-select mb-3">
        <option selected>Choose...</option>
        <option value="1">Option 1</option>
        <option value="2">Option 2</option>
    </select>

    <textarea class="form-control mb-3" rows="3"></textarea>
```

```
    <!-- Validation states -->
    <input class="form-control is-valid" type="text">
    <input class="form-control is-invalid" type="text">
    <div class="invalid-feedback">Please provide a valid value.</div>

    <!-- Floating labels -->
    <div class="form-floating mb-3">
        <input type="email" class="form-control" id="floatingInput"
               placeholder="name@example.com">
        <label for="floatingInput">Email address</label>
    </div>

    <button type="submit" class="btn btn-primary">Submit</button>
</form>
```

Modal Dialog:

```
<!-- Trigger -->
<button type="button" class="btn btn-primary"
        data-bs-toggle="modal" data-bs-target="#myModal">
    Open Modal
</button>

<!-- Modal -->
<div class="modal fade" id="myModal" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Modal Title</h5>
                <button type="button" class="btn-close"
                        data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <p>Modal body content here...</p>
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-secondary"
                        data-bs-dismiss="modal">Close</button>
                <button type="button" class="btn btn-primary">Save</button>
            </div>
        </div>
    </div>
</div>
```

Alerts:

```
<div class="alert alert-primary" role="alert">Primary alert!</div>
<div class="alert alert-success">Success message</div>
<div class="alert alert-danger">Error message</div>
<div class="alert alert-warning">Warning message</div>

<!-- Dismissible alert -->
<div class="alert alert-warning alert-dismissible fade show">
    <strong>Warning!</strong> Something needs attention.
    <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
</div>
```

3.5 Utility Classes

Bootstrap provides hundreds of utility classes for quickly applying styles without writing custom CSS. These are essential for rapid development.

Spacing Utilities (margin/padding):

Format: {property}{sides}-{breakpoint}-{size}

Property	Sides	Sizes
m = margin p = padding	t = top b = bottom s = start (left) e = end (right) x = left+right y = top+bottom (blank) = all	0 = 0 1 = 0.25rem 2 = 0.5rem 3 = 1rem 4 = 1.5rem 5 = 3rem auto

```
<!-- Spacing examples -->
<div class="m-3">Margin 1rem all sides</div>
<div class="mt-2">Margin top 0.5rem</div>
<div class="mb-5">Margin bottom 3rem</div>
<div class="mx-auto">Margin left + right auto (centers)</div>
<div class="py-3">Padding top + bottom 1rem</div>
<div class="ps-4">Padding start (left in LTR) 1.5rem</div>
<div class="m-md-5">Margin 3rem on medium+ screens</div>
```

Text Utilities:

```
<!-- Alignment -->
<p class="text-start">Left aligned</p>
<p class="text-center">Center aligned</p>
<p class="text-end">Right aligned</p>
<p class="text-md-end">Right on medium+ screens</p>

<!-- Transform -->
<p class="text-lowercase">LOWERCASE</p>
<p class="text-uppercase">uppercase</p>
<p class="text-capitalize">capitalize each word</p>

<!-- Weight & style -->
<p class="fw-bold">Bold</p>
<p class="fw-bolder">Bolder</p>
<p class="fw-normal">Normal</p>
<p class="fw-light">Light</p>
<p class="fst-italic">Italic</p>

<!-- Size -->
<p class="fs-1">Size 1 (largest)</p>
<p class="fs-6">Size 6</p>

<!-- Color -->
<p class="text-primary">Primary color</p>
<p class="text-success">Success</p>
<p class="text-muted">Muted</p>
<p class="text-white bg-dark">White on dark</p>

<!-- Wrapping & truncation -->
<p class="text-nowrap">Never wraps</p>
<p class="text-truncate" style="width: 100px;">Long text truncated...</p>
<p class="text-break">Long-word-that-breaks</p>
```

Background & Border Utilities:

```
<!-- Backgrounds -->
<div class="bg-primary text-white">Primary background</div>
<div class="bg-success">Success</div>
<div class="bg-light">Light</div>
<div class="bg-dark text-white">Dark</div>
```

```
<div class="bg-transparent">Transparent</div>

<!-- Background opacity -->
<div class="bg-primary bg-opacity-75">75% opacity</div>
<div class="bg-primary bg-opacity-50">50% opacity</div>

<!-- Borders -->
<div class="border">All borders</div>
<div class="border-top">Top only</div>
<div class="border-end">Right only (LTR)</div>
<div class="border-0">No border</div>
<div class="border-primary">Primary color border</div>
<div class="border-2">Thicker border</div>

<!-- Rounded -->
<div class="rounded">Rounded corners</div>
<div class="rounded-circle">Circle (for square images)</div>
<div class="rounded-pill">Pill shape</div>
<div class="rounded-3">Larger radius</div>
<div class="rounded-top">Only top rounded</div>
```

Display & Flex Utilities:

```
<!-- Display -->
<div class="d-none">Hidden</div>
<div class="d-block">Block</div>
<div class="d-inline">Inline</div>
<div class="d-inline-block">Inline block</div>
<div class="d-flex">Flex container</div>
<div class="d-grid">Grid container</div>

<!-- Responsive -->
<div class="d-none d-md-block">Hidden on mobile, visible on md+</div>
<div class="d-md-none">Visible on mobile, hidden on md+</div>

<!-- Flex utilities -->
<div class="d-flex justify-content-between align-items-center">
    <div>Left</div>
    <div>Right</div>
</div>

<div class="d-flex flex-column">
    <!-- Vertical flex -->
</div>

<div class="d-flex flex-wrap gap-2">
    <!-- Items with gap and wrap -->
</div>

<!-- Justify content options -->
<!-- justify-content-{start, end, center, between, around, evenly} -->
<!-- align-items-{start, end, center, baseline, stretch} -->
<!-- flex-{row, column, row-reverse, column-reverse} -->
<!-- flex-{wrap, nowrap, wrap-reverse} -->
```

Sizing Utilities:

```
<!-- Width -->
<div class="w-25">25% width</div>
<div class="w-50">50% width</div>
<div class="w-75">75% width</div>
<div class="w-100">100% width</div>
<div class="w-auto">Auto width</div>
<div class="mw-100">Max-width 100%</div>
```

```
<!-- Height -->
<div class="h-25">25% height</div>
<div class="h-100">Full height</div>
<div class="mh-100">Max-height 100%</div>

<!-- Viewport -->
<div class="vh-100">100% viewport height</div>
<div class="vw-100">100% viewport width</div>
```

Position Utilities:

```
<div class="position-static">Static (default)</div>
<div class="position-relative">Relative</div>
<div class="position-absolute">Absolute</div>
<div class="position-fixed">Fixed</div>
<div class="position-sticky top-0">Sticky to top</div>

<!-- Top/right/bottom/left -->
<div class="position-absolute top-0 start-0">Top-left</div>
<div class="position-absolute top-50 start-50 translate-middle">Centered</div>
```

3.6 Customization

Bootstrap can be customized via CSS variables (easy) or by compiling Sass (powerful).

Method 1: CSS Variables Override:

```
<style>
/* Override Bootstrap variables */
:root {
    --bs-primary: #ff5722;
    --bs-primary-rgb: 255, 87, 34;
    --bs-body-font-family: 'Roboto', sans-serif;
    --bs-body-font-size: 1rem;
    --bs-border-radius: 0.5rem;
}

/* Custom styles after Bootstrap */
.btn-primary {
    background-color: var(--bs-primary);
    border-color: var(--bs-primary);
}
</style>
```

Method 2: Sass Customization (Advanced):

```
// custom.scss
// 1. Override defaults
$primary: #ff5722;
$font-family-sans-serif: 'Roboto', sans-serif;
$border-radius: 0.5rem;
$enable-shadows: true;

// 2. Import Bootstrap
@import "node_modules/bootstrap/scss/bootstrap";

// 3. Add custom styles
.custom-card {
    @extend .card;
    border: 2px solid $primary;
}
```

PART 4: INTERVIEW QUESTIONS & ANSWERS

This section contains the most commonly asked HTML, CSS, and Bootstrap interview questions for Associate Software Engineer positions. Questions are organized by level (L1 = entry/junior, L2 = intermediate) and topic.

4.1 L1 (Basic/Junior) Interview Questions

HTML Questions:

Q1. What is HTML and why is it used?

HTML (HyperText Markup Language) is the standard markup language used to create the structure of web pages. It is NOT a programming language but a markup language that uses tags to describe how content should be displayed. HTML is used to: define the structure of web pages, embed images, audio, and videos, create forms and links, ensure accessibility and SEO. Every web page on the internet uses HTML as its foundation.

Q2. What is the difference between HTML and HTML5?

HTML5 is the latest major version of HTML with several improvements: **Doctype** - HTML5 uses simple <!DOCTYPE html> vs complex DTD in HTML4. **New semantic tags** - header, footer, article, section, nav, aside. **Multimedia** - Native <audio> and <video> tags (no plugins needed). **New form input types** - email, date, color, range, etc. **APIs** - Canvas, LocalStorage, Geolocation, Web Workers. **Better error handling** and SVG support.

Q3. What are HTML tags and attributes?

Tags are keywords enclosed in angle brackets that define HTML elements. Most tags come in pairs (opening and closing). **Attributes** provide additional information about elements and are written in the opening tag as name="value" pairs. Example: <img src="photo.jpg" alt="My Photo" width="200"> - Here img is the tag, and src, alt, width are attributes.

Q4. What is the difference between and ?

** ** is a block-level element that takes the full width of its container and starts on a new line. It's used to group block-level content. **** is an inline element that only takes as much width as needed and doesn't break the line. It's used to group inline content for styling purposes. Both are generic containers without semantic meaning—use semantic tags (article, section, etc.) when possible.

Q5. What are void/self-closing elements? Give examples.

Void elements are HTML elements that don't have a closing tag and cannot contain any content. They are self-closing. Examples: **<img>**, **
, **<hr>, **<input>**, **<meta>**, **<link>**, **<area>**, **<base>**, **<col>**, **<source>**, **<track>**, **<wbr>**, **<embed>**, **<param>**.

Q6. What is semantic HTML?

Semantic HTML uses tags that convey meaning about the content, not just appearance. Examples are <header>, <nav>, <main>, <article>, <section>, <aside>, <footer>. Benefits: **Accessibility** - Screen readers understand structure. **SEO** - Search engines rank content better. **Maintainability** - Code is self-documenting. **Cleaner code** - Less reliance on classes/IDs.

Q7. Why is the alt attribute important for images?

The alt (alternative text) attribute is crucial for: **Accessibility** - Screen readers read alt text to visually impaired users. **Fallback** - Shows text if image fails to load. **SEO** - Search engines use alt text to understand image content. **Context** - Provides context when images are turned off. Always include meaningful alt text; for decorative images, use empty alt="" so screen readers skip them.

Q8. What is the difference between GET and POST in forms?

GET: Data sent in URL (visible), limited size (~2048 chars), can be bookmarked/cached, should be used for retrieving data, not secure for sensitive info. **POST**: Data sent in request body (not visible in URL), no size limit, cannot be bookmarked, used for submitting data (forms, file uploads), more secure for sensitive data like passwords. Rule of thumb: GET for reading data, POST for changing/submitting data.

Q9. What is the purpose of the <!DOCTYPE> declaration?

<!DOCTYPE html> tells the browser which version of HTML the document uses. It must be the very first line of the document. Without it, browsers enter "quirks mode" which can cause inconsistent rendering. HTML5 uses the simplified <!DOCTYPE html>, while older versions had longer DTD declarations.

Q10. What is the difference between id and class attributes?

id: Must be unique within a page (only one element can have a specific id). Used in CSS as #idname, in JS as getElementById(). Higher CSS specificity (100). Used for anchor links (#section). **class**: Can be applied to multiple elements; an element can have multiple classes. Used in CSS as .classname, in JS as getElementsByClassName() or querySelector(). Lower specificity (10). Better for styling reusable components.

CSS Questions:

Q11. What is CSS and what are the different ways to apply it?

CSS (Cascading Style Sheets) describes the presentation of HTML documents. Three ways to apply: **1. Inline CSS** - style attribute on elements (highest specificity, hardest to maintain). **2. Internal CSS** - <style> tag inside <head> (good for single-page styles). **3. External CSS** - separate .css file linked via <link> tag (best practice, reusable, cacheable, separates concerns).

Q12. Explain the CSS Box Model.

Every HTML element is a rectangular box made of 4 parts: **Content** (innermost, holds text/images), **Padding** (space between content and border), **Border** (line around padding), **Margin** (outermost, space between elements). By default (box-sizing: content-box), width/height applies to content only, so total size = content + padding + border + margin. Using box-sizing: border-box, width/height includes padding and border (much easier to work with).

Q13. What is the difference between margin and padding?

Margin is space OUTSIDE the element's border. It pushes other elements away. Margins can be negative and can collapse (when vertical margins meet, the larger one wins). **Padding** is space INSIDE the element, between content and border. Padding cannot be negative. Padding is part of the element (clickable, takes background color), margin is not.

Q14. What is CSS specificity and how is it calculated?

Specificity determines which CSS rule wins when multiple rules target the same element. Calculated as four numbers (a,b,c,d): **a** = inline styles (1 if yes, else 0), **b** = number of IDs, **c** = number of classes, attributes, pseudo-classes, **d** = number of elements, pseudo-elements. Example: #header .btn:hover has specificity (0,1,2,0). Higher specificity wins. !important overrides all (avoid using it).

Q15. What are pseudo-classes and pseudo-elements?

Pseudo-classes select elements based on state or condition. Single colon: :hover, :focus, :active, :first-child, :nth-child(n), :not(), :checked. Example: a:hover changes appearance on hover. **Pseudo-elements** style specific parts of an element. Double colon (in modern CSS): ::before, ::after, ::first-letter, ::first-line, ::selection, ::placeholder. Example: p::first-letter styles only the first letter.

Q16. What is the difference between display: none and visibility: hidden?

display: none: Element is completely removed from the layout. It takes no space, and screen readers ignore it. The page rearranges as if the element doesn't exist. **visibility: hidden**: Element is hidden visually but still occupies its space in the layout. Other elements don't move. Screen readers may still announce it (use aria-hidden too). **opacity: 0**: Invisible but still clickable and takes space.

Q17. Explain the difference between em, rem, px, and %.

px: Absolute fixed pixels - doesn't scale. **em**: Relative to the PARENT element's font-size (2em = 2x parent's font size). Can compound in nested elements (problem!). **rem**: Relative to the ROOT (html) element's font-size. Most predictable - recommended for responsive design. **%**: Relative to the parent's value for that property (e.g., width: 50% = half of parent's width). Use rem for fonts and spacing, % for widths, px for borders.

Q18. What is the difference between inline, block, and inline-block?

inline: Takes only needed width, doesn't break line. Cannot set width/height. Examples: , <a>, . **block**: Takes full width, starts on new line. Width/height work. Examples: , <p>, <h1>. **inline-block**: Best of both - sits inline like inline elements, but you can set width/height like block elements. Useful for buttons, navigation items.

Q19. What are CSS selectors? Name different types.

CSS selectors target HTML elements for styling. Types: **Universal** (*), **Element** (p, div), **Class** (.classname), **ID** (#idname), **Attribute** ([type="text"]), **Descendant** (div p), **Child** (div > p), **Adjacent sibling** (h1 + p), **General sibling** (h1 ~ p), **Pseudo-class** (a:hover), **Pseudo-element** (p::before), **Grouping** (h1, h2, h3).

Q20. How do you center a div horizontally and vertically?

Modern way (Flexbox): Set parent to display: flex; justify-content: center (horizontal); align-items: center (vertical). Grid: display: grid; place-items: center. Old way (absolute positioning): position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%). For horizontal only: margin: 0 auto on a block element with a defined width.

Bootstrap Questions:

Q21. What is Bootstrap and why is it popular?

Bootstrap is a free, open-source CSS framework for building responsive, mobile-first websites. Created by Twitter, now maintained as Bootstrap 5. **Popular because:** Mobile-first responsive design, pre-built components (buttons, forms, modals), powerful 12-column grid system, cross-browser compatibility, customizable via Sass/CSS variables, excellent documentation, large community, saves development time significantly.

Q22. Explain the Bootstrap grid system.

Bootstrap uses a 12-column responsive grid built on Flexbox. Three required components: **.container** (or .container-fluid) wraps the grid, **.row** creates a horizontal row, **.col-*** defines columns inside rows. Columns can be specified per breakpoint: col-12 (mobile), col-md-6 (tablet 50%), col-lg-4 (desktop 33%). Total columns should equal 12 per row. Six breakpoints: xs (<576px), sm (≥576), md (≥768), lg (≥992), xl (≥1200), xxl (≥1400).

Q23. What is the difference between .container and .container-fluid?

.container: Has a fixed max-width that changes at each breakpoint (e.g., 540px on sm, 720px on md, etc.). It's centered with auto margins. Use when you want bounded content with whitespace on the sides on large screens. **.container-fluid**: 100% width at all screen sizes, no max-width. Use when you want content to span the full screen width (like for full-width hero sections or admin dashboards).

Q24. How do you make a Bootstrap navbar responsive?

Use .navbar with .navbar-expand-{breakpoint} class. The navbar collapses on screens smaller than the specified breakpoint. Add a .navbar-toggler button with data-bs-toggle="collapse" and data-bs-target pointing to the collapsible menu (.navbar-collapse). Example: <nav class="navbar navbar-expand-lg"> will collapse below 992px. The toggler shows a hamburger menu icon on smaller screens.

Q25. What are Bootstrap utility classes?

Utility classes are single-purpose classes that apply specific styles. They allow you to style elements without writing custom CSS. Categories: **Spacing** (m-3, p-2, mt-4, mx-auto), **Text** (text-center, text-primary, fw-bold), **Display** (d-flex, d-none, d-md-block), **Flex** (justify-content-between, align-items-center), **Borders** (border, rounded), **Colors** (bg-primary, text-success). They speed up development and ensure consistency.

4.2 L2 (Intermediate) Interview Questions

Advanced HTML Questions:

Q26. What is the difference between <script>, <script async>, and <script defer>?

<script> (default): Blocks HTML parsing while script is downloaded AND executed. Worst for performance. **<script async>**: Downloads script in parallel with HTML parsing, then pauses parsing to execute when ready. Order of execution is not guaranteed. Good for independent scripts (analytics). **<script defer>**: Downloads in parallel, but waits to execute until HTML parsing is complete. Maintains script order. Best for scripts that depend on DOM or each other. Both async and defer only work on external scripts.

Q27. What is the difference between localStorage, sessionStorage, and cookies?

localStorage: ~5-10MB capacity, persists until manually cleared, NOT sent with HTTP requests, same-origin only, synchronous API. Good for: user preferences, cached data. **sessionStorage**: Same as localStorage but cleared when tab closes. Good for: form data, temporary state. **Cookies**: ~4KB max, sent with EVERY HTTP request, can be set with expiration, supports HttpOnly (XSS protection), Secure flag, SameSite. Good for: authentication tokens, server-needed data. Cookies impact performance because they're sent on every request.

Q28. Explain the difference between <article>, <section>, and .

<article>: Self-contained, independently distributable content (blog post, news article, forum post, product card). Should make sense on its own. **<section>**: Thematic grouping of content, typically with a heading. Represents a section of a document. Use when content shares a common theme. ** **: Generic container with no semantic meaning. Use ONLY when no semantic element fits, mainly for styling/layout purposes.

Q29. What is the purpose of meta viewport tag?

<meta name="viewport" content="width=device-width, initial-scale=1.0"> controls how the page is displayed on mobile devices. **width=device-width**: Sets width to follow device's screen width (instead of default 980px). **initial-scale=1.0**: Sets initial zoom level. Without this tag, mobile browsers render at desktop width then zoom out, making text tiny. Essential for responsive design—a must in any modern HTML page.

Q30. What are data-* attributes and when do you use them?

data-* attributes are custom HTML attributes prefixed with 'data-' that allow you to store extra information on standard HTML elements without using non-standard attributes. Used for: **JavaScript interaction** - access via element.dataset.attributeName (data-user-id becomes dataset.userId). **CSS styling** - select with [data-state="active"]. **Framework integration** - Bootstrap uses data-bs-toggle, data-bs-target for components like modals and dropdowns. Example: <button data-user-id="123">.

Advanced CSS Questions:

Q31. Explain Flexbox in detail with main properties.

Flexbox is a one-dimensional layout system. **Container properties**: display: flex (creates flex container), flex-direction (row/column/reverse), flex-wrap (wrap/nowrap), justify-content (alignment on main axis: flex-start, center, space-between, space-around, space-evenly), align-items (alignment on cross axis: stretch, center, flex-start, flex-end, baseline), align-content (multi-line alignment), gap. **Item properties**: flex-grow (how much to grow), flex-shrink (how much to shrink), flex-basis (initial size), shorthand flex: 1 1 auto, order (reorder items), align-self (override container's align-items).

Q32. What is the difference between Flexbox and Grid?

Flexbox: One-dimensional layout (either rows OR columns at a time). Content-first design - layout adapts to content size. Best for: navbars, card layouts, centering, small components. **Grid**: Two-dimensional layout (rows AND columns simultaneously). Layout-first design - content fits into defined cells. Best for: page layouts, image galleries, dashboards, complex layouts. They can be used together - Grid for page structure, Flexbox for components within. Browser support: both have excellent support in modern browsers.

Q33. What is the difference between absolute, relative, fixed, and sticky positioning?

static: Default. Normal document flow. **relative**: Positioned relative to its normal position. Still takes original space. Creates positioning context for absolute children. **absolute**: Removed from flow, positioned relative to nearest POSITIONED ancestor (or <html> if none). **fixed**: Removed from flow, positioned relative to VIEWPORT. Stays in place during scroll (e.g., sticky headers). **sticky**: Hybrid - acts like relative until it hits a threshold (e.g., top: 0), then acts like fixed. Great for sticky table headers, section headings.

Q34. What is BEM methodology in CSS?

BEM (Block Element Modifier) is a CSS naming convention for writing maintainable code. **Block**: Independent component (.card, .button). **Element**: Part of a block, separated by __ (.card__title, .card__image). **Modifier**: Variation of block or element, separated by -- (.card--featured, .button--large). Benefits: clear hierarchy, no specificity issues, reusable components, prevents CSS conflicts. Example: <h2 class="card__title card__title--large">.

Q35. Explain CSS Box Sizing and the difference between content-box and border-box.

box-sizing controls how width and height are calculated. **content-box** (default): width and height apply to CONTENT only. Total width = width + padding + border. So width: 200px + padding: 20px + border: 2px = 244px actual width. Confusing and error-prone. **border-box**: width and height include padding and border. width: 200px means actual width is 200px (padding and border are subtracted from content). Most developers apply this globally: *, *::before, *::after { box-sizing: border-box; }. Predictable and easier to work with.

Q36. What is the difference between transition and animation in CSS?

Transition: Smoothly changes property values between two states. Requires a trigger (hover, focus, class change). Simple - just from current to target value. Cannot loop or have multiple keyframes. Best for hover effects. **Animation**: Uses @keyframes for complex multi-step animations. Doesn't need a trigger - can run on page load. Supports infinite loops, multiple keyframes (0%, 25%, 50%, 100%), pause/play states, reverse direction. Best for loading spinners, complex sequences.

Q37. What are CSS custom properties (variables) and how are they useful?

CSS custom properties (--name) are variables that can be reused throughout CSS. Declared with -- and accessed with var(). Example: :root { --primary: #1565c0; } then color: var(--primary). Benefits: **Reusability** - DRY principle. **Theming** - easy dark/light mode by overriding variables. **Dynamic** - can be changed with JavaScript (element.style.setProperty('--color', 'red')). **Cascading** - inherit through the cascade. **Scoping** - can be limited to specific selectors. Unlike Sass variables, they exist at runtime.

Q38. How do media queries work and what is mobile-first design?

Media queries apply CSS based on device characteristics. Syntax: @media (min-width: 768px) { ... }. Common conditions: min-width/max-width, orientation, prefers-color-scheme (dark mode), hover capability. **Mobile-first** approach: Write base CSS for mobile, then use min-width media queries to add styles for larger screens. Advantages: smaller default CSS for mobile (performance), progressive enhancement, easier to maintain. Desktop-first uses max-width and is generally considered outdated.

Q39. What is z-index and how does stacking context work?

z-index controls vertical stacking order of POSITIONED elements (position: relative, absolute, fixed, sticky). Higher z-index appears on top. Default is auto (0). **Stacking context** is created by: root element, positioned element with z-index ≠ auto, opacity less than 1, transform, filter, isolation: isolate, position: fixed/sticky. Within a stacking context, child z-indexes are local. Common issue: child with high z-index can't escape parent's stacking context. To fix: increase parent's z-index or remove the stacking context property.

Q40. What is the difference between visibility: hidden, display: none, and opacity: 0?

display: none: Element removed from DOM layout entirely. No space reserved. Not accessible to screen readers. Not focusable. Animations don't work on it. **visibility: hidden**: Element invisible but space is reserved. Not interactive (can't click). Children can override with visibility: visible. **opacity: 0**: Invisible but takes space. STILL INTERACTIVE (can be clicked, focused). Can be transitioned/animated. Good for fade effects. Choose based on whether you want to preserve space and interactivity.

Advanced Bootstrap Questions:

Q41. How do you customize Bootstrap?

Three methods: **1. CSS Override** - Write custom CSS after Bootstrap to override styles (simplest, increase specificity if needed). **2. CSS Variables** - Bootstrap 5 uses CSS variables (--bs-primary, --bs-body-font-size). Override them in :root for global changes. **3. Sass Customization** (most powerful) - Install Bootstrap via npm, override Sass variables BEFORE importing Bootstrap's source, then compile. This recompiles Bootstrap with your customizations. Can selectively import only needed components for smaller file size.

Q42. What are Bootstrap breakpoints and how do they work?

Bootstrap 5 has 6 breakpoints (min-width based, mobile-first): **xs** <576px (default, no prefix), **sm** ≥576px (.col-sm-*), **md** ≥768px (.col-md-*), **lg** ≥992px (.col-lg-*), **xl** ≥1200px (.col-xl-*), **xxl** ≥1400px (.col-xxl-*). Classes apply at their breakpoint AND ABOVE. Example: col-md-6 means 50% width on medium screens (768px) and all larger screens, unless overridden by larger breakpoint class. Use multiple classes for different sizes: col-12 col-md-6 col-lg-4.

Q43. Explain the difference between col-md-6 and col-md-offset-6 (in Bootstrap 5: offset-md-6).

col-md-6: Column takes 6 out of 12 columns (50% width) on medium screens and up. Element is placed where it normally goes. **offset-md-6**: Pushes the column to the right by 6 columns on medium screens. Used to create gaps without adding empty columns. Example: centers a 4-column wide element by offsetting it by 4 columns. Note: In Bootstrap 5, you can also use flexbox utilities like .ms-auto (margin-start: auto) for similar effects.

Q44. How does the Bootstrap grid handle responsiveness?

Bootstrap's grid uses Flexbox and a mobile-first approach. Classes are cumulative - smaller breakpoint classes apply unless overridden. Example: means: 12 columns wide on mobile (full width), 6 on tablet (50%), 4 on desktop (33%). At each breakpoint, the previous setting cascades up unless changed. You can also use auto-layout (.col without number) where columns share space equally, or .col-auto for content-width columns.

Q45. What is the purpose of .row in Bootstrap and why must columns be inside it?

The .row class: **Creates flex container** for columns. **Applies negative margins** (-15px on Bootstrap 4, custom in 5 via --bs-gutter-x) to counteract column padding for proper alignment. **Ensures correct gutters** (spacing between columns). **Manages column wrapping**. Without .row, columns will have

incorrect spacing and break layout. Always wrap columns in .row, and always put .row inside .container or .container-fluid.

Q46. How would you create a responsive image in Bootstrap?

Use the **.img-fluid** class on the <img> tag. It applies max-width: 100% and height: auto, so the image scales with its container. Example: <img src="photo.jpg" class="img-fluid" alt="Description">. For circular images: .rounded-circle. For thumbnail border: .img-thumbnail. For art direction (different images at different breakpoints), use HTML5 <picture> element with media queries.

Q47. What are Bootstrap components that require JavaScript?

Components requiring Bootstrap's JavaScript bundle: **Modal** dialogs, **Dropdowns**, **Collapsible** elements (accordions, navbar toggle), **Tooltips**, **Popovers**, **Carousel**, **Toasts**, **Tabs**, **Offcanvas**, **Scrollspy**. These are activated via data-bs-toggle and data-bs-target attributes. Bootstrap 5 removed jQuery dependency and uses vanilla JavaScript. Include the bootstrap.bundle.min.js (which includes Popper.js needed for dropdowns/tooltips/popovers).

Q48. How does Bootstrap implement dark mode (in v5.3+)?

Bootstrap 5.3 introduced built-in dark mode via the data-bs-theme attribute. Set on html or any element: <html data-bs-theme="dark">. Components automatically adjust colors using CSS variables. You can have dark sections on a light page: . Toggle via JavaScript: document.documentElement.setAttribute('data-bs-theme', 'dark'). Custom themes can be created by overriding CSS variables for [data-bs-theme="custom"].

Q49. What are the advantages and disadvantages of using Bootstrap?

Advantages: Fast development, responsive out of the box, cross-browser compatibility, excellent documentation, large community, mobile-first, customizable, accessible components, consistent design. **Disadvantages**: Sites may look generic ("Bootstrap look"), large file size if all components imported, learning curve for customization, may include unused code (mitigate with PurgeCSS or selective imports), can lead to CSS class bloat in HTML, harder to deviate from default design system. Solution: Use only what you need via Sass.

Q50. How do you optimize a Bootstrap site for performance?

1. Import only needed components via Sass (don't include entire framework). **2. Use PurgeCSS or similar** to remove unused CSS classes. **3. Use the minified versions** (.min.css, .min.js). **4. Use a CDN** for caching benefits. **5. Defer JavaScript** loading where possible. **6. Use HTTP/2** with proper caching headers. **7. Compress images** and use modern formats (WebP). **8. Replace heavy components** with lighter custom solutions if used only once. **9. Avoid loading unused fonts/icons**. **10. Enable Gzip/Brotli** compression on server.

4.3 Coding Challenges (Practical Examples)

These are common practical coding tasks asked during L1/L2 interviews. Practice writing them from memory until you can do them fluently. Each example focuses on a real-world UI pattern that demonstrates your understanding of HTML structure, CSS techniques, and Bootstrap usage.

Challenge 1: Build a Responsive Navbar (Pure CSS)

Create a navbar that converts to a hamburger menu on mobile screens without using JavaScript.

```
<!-- HTML -->
<nav class="navbar">
  <div class="logo">MyBrand</div>
  <input type="checkbox" id="menu-toggle" />
  <label for="menu-toggle" class="hamburger">&#9776;</label>
  <ul class="nav-links">
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Services</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>

/* CSS */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: #1565c0;
  color: white;
}
.logo { font-size: 1.5rem; font-weight: bold; }
.nav-links {
  display: flex;
  list-style: none;
  gap: 2rem;
}
.nav-links a { color: white; text-decoration: none; }
.hamburger { display: none; font-size: 1.5rem; cursor: pointer; }
#menu-toggle { display: none; }

@media (max-width: 768px) {
  .hamburger { display: block; }
  .nav-links {
    display: none;
    flex-direction: column;
    width: 100%;
    position: absolute;
    top: 60px; left: 0;
    background: #1565c0;
    padding: 1rem;
  }
  #menu-toggle:checked ~ .nav-links { display: flex; }
}
```

Challenge 2: Center a Modal Dialog (3 Methods)

A classic interview question. Show multiple techniques to demonstrate depth.

```
/* Method 1: Flexbox (modern, recommended) */
```

```
.modal-overlay {
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,0.5);
  display: flex;
  justify-content: center;
  align-items: center;
}
.modal { background: white; padding: 2rem; border-radius: 8px; }

/* Method 2: CSS Grid */
.modal-overlay {
  position: fixed;
  inset: 0;
  display: grid;
  place-items: center;
}

/* Method 3: Transform (classic) */
.modal {
  position: fixed;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%);
}
```

Challenge 3: Pure CSS Dropdown Menu

```
<!-- HTML -->
<ul class="menu">
  <li class="dropdown">
    Products &#9660;
    <ul class="submenu">
      <li><a href="#">Web Apps</a></li>
      <li><a href="#">Mobile Apps</a></li>
      <li><a href="#">APIs</a></li>
    </ul>
  </li>
</ul>

/* CSS */
.menu { list-style: none; padding: 0; }
.dropdown {
  position: relative;
  display: inline-block;
  padding: 10px 20px;
  cursor: pointer;
  background: #1565c0;
  color: white;
}
.submenu {
  display: none;
  position: absolute;
  top: 100%;
  left: 0;
  list-style: none;
  padding: 0;
  background: white;
  box-shadow: 0 2px 8px rgba(0,0,0,0.15);
  min-width: 160px;
}
.dropdown:hover .submenu { display: block; }
.submenu li { padding: 10px 20px; }
.submenu li:hover { background: #e3f2fd; }
.submenu a { color: #212121; text-decoration: none; }
```

Challenge 4: Responsive Bootstrap Card Grid

Build a card layout: 1 column on mobile, 2 on tablet, 3 on desktop, 4 on large screens.

```
<div class="container py-4">
  <div class="row g-4">
    <!-- Repeat this column block for each card -->
    <div class="col-12 col-md-6 col-lg-4 col-xl-3">
      <div class="card h-100 shadow-sm">
        <img src="image.jpg" class="card-img-top" alt="...">
        <div class="card-body">
          <h5 class="card-title">Product Name</h5>
          <p class="card-text">Short description here.</p>
          <a href="#" class="btn btn-primary">View Details</a>
        </div>
        <div class="card-footer text-muted">
          <small>Updated 3 mins ago</small>
        </div>
      </div>
    </div>
    <!-- More cards... -->
  </div>
</div>
```

Key concepts: col-12 (mobile-first full width), responsive breakpoints (md/lg/xl), g-4 for gutter spacing, h-100 to make all cards equal height in a row.

Challenge 5: Holy Grail Layout with CSS Grid

The classic page layout: header, footer, two sidebars, and main content area.

```
/* CSS */
.layout {
  display: grid;
  min-height: 100vh;
  grid-template-areas:
    "header  header  header"
    "nav     main    aside"
    "footer  footer  footer";
  grid-template-rows: auto 1fr auto;
  grid-template-columns: 200px 1fr 200px;
  gap: 10px;
}
header { grid-area: header; background: #1565c0; }
nav    { grid-area: nav;    background: #e3f2fd; }
main   { grid-area: main;   background: #ffffff; }
aside  { grid-area: aside;  background: #e3f2fd; }
footer { grid-area: footer; background: #424242; }

/* Make it responsive */
@media (max-width: 768px) {
  .layout {
    grid-template-areas:
      "header"
      "nav"
      "main"
      "aside"
      "footer";
    grid-template-columns: 1fr;
  }
}
```

Challenge 6: Form with HTML5 + CSS Validation Styling

```
<!-- HTML -->
<form class="contact-form" novalidate>
  <label>
    Email:
    <input type="email" required placeholder="you@example.com" />
    <span class="error">Please enter a valid email</span>
  </label>
  <label>
    Password:
    <input type="password" required minlength="8" />
    <span class="error">Minimum 8 characters required</span>
  </label>
  <button type="submit">Submit</button>
</form>

/* CSS */
.contact-form input {
  width: 100%;
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.contact-form input:focus {
  outline: none;
  border-color: #1565c0;
  box-shadow: 0 0 0 3px rgba(21,101,192,0.2);
}
/* :invalid only after user interacts */
.contact-form input:not(:placeholder-shown):invalid {
  border-color: #d32f2f;
}
.contact-form input:not(:placeholder-shown):valid {
  border-color: #388e3c;
}
.error { display: none; color: #d32f2f; font-size: 0.85em; }
.contact-form input:not(:placeholder-shown):invalid + .error {
  display: block;
}
```

Challenge 7: Pure CSS Loading Spinner

```
/* HTML */
<div class="spinner"></div>

/* CSS */
.spinner {
  width: 50px;
  height: 50px;
  border: 5px solid #e3f2fd;
  border-top-color: #1565c0;
  border-radius: 50%;
  animation: spin 1s linear infinite;
}
@keyframes spin {
  to { transform: rotate(360deg); }
}

/* Bonus: Dots loader */
.dots {
  display: inline-flex;
  gap: 6px;
}
```

```
.dots span {
  width: 10px;
  height: 10px;
  background: #1565c0;
  border-radius: 50%;
  animation: bounce 1.4s infinite ease-in-out;
}
.dots span:nth-child(2) { animation-delay: 0.2s; }
.dots span:nth-child(3) { animation-delay: 0.4s; }
@keyframes bounce {
  0%, 80%, 100% { transform: scale(0); }
  40% { transform: scale(1); }
}
```

Challenge 8: Mobile-First Responsive Layout

Build a feature section that stacks on mobile and becomes a 3-column layout on desktop. Mobile-first means: write base styles for mobile, then add larger breakpoints.

```
/* Mobile-first: base styles target small screens */
.features {
  display: flex;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
}
.feature {
  background: #f5f5f5;
  padding: 1.5rem;
  border-radius: 8px;
}
.feature h3 { font-size: 1.2rem; margin-bottom: 0.5rem; }
.feature p { color: #616161; font-size: 0.95rem; }

/* Tablet: 768px and up */
@media (min-width: 768px) {
  .features {
    flex-direction: row;
    flex-wrap: wrap;
  }
  .feature { flex: 1 1 calc(50% - 1rem); }
}

/* Desktop: 1024px and up */
@media (min-width: 1024px) {
  .features { padding: 2rem; }
  .feature { flex: 1 1 calc(33.333% - 1rem); }
  .feature h3 { font-size: 1.5rem; }
}
```

Challenge 9: Image Gallery with Hover Effects

```
<!-- HTML -->
<div class="gallery">
  <figure class="gallery-item">
    <img src="img1.jpg" alt="Photo 1" />
    <figcaption>Beautiful Sunset</figcaption>
  </figure>
  <!-- More figures... -->
</div>

/* CSS */
.gallery {
```

```
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 1rem;
  padding: 1rem;
}
.gallery-item {
  position: relative;
  overflow: hidden;
  border-radius: 8px;
  margin: 0;
  cursor: pointer;
}
.gallery-item img {
  width: 100%;
  height: 250px;
  object-fit: cover;
  transition: transform 0.4s ease;
}
.gallery-item:hover img {
  transform: scale(1.1);
}
.gallery-item figcaption {
  position: absolute;
  bottom: 0; left: 0; right: 0;
  background: linear-gradient(transparent, rgba(0,0,0,0.8));
  color: white;
  padding: 1rem;
  transform: translateY(100%);
  transition: transform 0.3s ease;
}
.gallery-item:hover figcaption {
  transform: translateY(0);
}
```

Challenge 10: Sticky Footer Pattern

Make the footer stick to the bottom of the viewport when content is short, but flow normally when content is long. A very common requirement.

```
/* Method 1: Flexbox (recommended) */
body {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  margin: 0;
}
main { flex: 1; }
footer { background: #212121; color: white; padding: 1rem; }

/* Method 2: CSS Grid */
body {
  min-height: 100vh;
  display: grid;
  grid-template-rows: auto 1fr auto;
  margin: 0;
}

/* HTML structure */
<body>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
</body>
```

Final Interview Tips

Before the Interview

- **Practice writing code on paper or whiteboard** - many interviews still test this skill
- **Build small projects**: a landing page, a navbar, a card layout, a form - keep them in a portfolio
- **Understand 'why', not just 'how'**: know why flexbox is one-dimensional or why mobile-first matters
- **Learn browser DevTools** - inspect element, computed styles, responsive view, console
- **Read the documentation**: MDN for HTML/CSS, getbootstrap.com for Bootstrap
- **Practice on platforms**: Frontend Mentor, CodePen challenges, CSS Battle

During the Interview

- **Think aloud** - explain your thought process; interviewers care about reasoning
- **Ask clarifying questions** before coding (browser support? mobile-first? accessibility?)
- **Start simple**, then improve - show iterative thinking
- **Comment your code** to show structure and intent
- **Mention alternatives**: 'I'd use flexbox here, but grid could also work because...'
- **Discuss trade-offs**: performance, accessibility, browser support, maintainability
- **If stuck, say so**: 'I'd typically look up the exact syntax, but the approach would be...'

Common Mistakes to Avoid

- Using **div for everything** instead of semantic HTML (use header, nav, main, article, footer)
- Forgetting the **viewport meta tag** in HTML head for responsive design
- Not setting **alt attributes** on images (accessibility and SEO)
- Using **!important** as a fix - it's a sign of poor specificity management
- **Fixed pixel widths** instead of responsive units (use rem, em, %, vw/vh)
- Ignoring **accessibility** - labels for inputs, sufficient color contrast, keyboard navigation
- Loading **entire Bootstrap** when you only need a few components
- Not testing on **real mobile devices** - emulators only go so far

Recommended Learning Path

- **Week 1-2**: HTML fundamentals, all tags, forms, semantic HTML, accessibility basics
- **Week 3-4**: CSS basics - selectors, box model, display, positioning, units, colors, typography
- **Week 5-6**: CSS layout - flexbox deeply, then grid, then responsive design with media queries
- **Week 7**: CSS advanced - transitions, animations, transforms, variables, methodologies (BEM)
- **Week 8**: Bootstrap 5 - grid system, components, utilities, customization
- **Week 9-10**: Build 3 projects - landing page, dashboard layout, multi-page site with Bootstrap
- **Week 11-12**: Practice interview questions daily, mock interviews, build portfolio

Useful Resources

- **MDN Web Docs** - developer.mozilla.org (the authoritative reference for HTML/CSS)
- **Bootstrap Docs** - getbootstrap.com (official documentation)
- **CSS-Tricks** - css-tricks.com (in-depth CSS articles and guides)
- **Flexbox Froggy / Grid Garden** - interactive games for learning layout
- **Frontend Mentor** - real-world coding challenges with designs
- **Can I Use** - caniuse.com (browser compatibility checker)
- **CodePen** - codepen.io (try snippets live, browse community examples)
- **W3Schools** - w3schools.com (good for quick syntax lookups)

All the Best for Your Interview!

Remember: technical knowledge is only half the battle. Communication, problem-solving approach, and the ability to learn quickly are equally valuable. Stay calm, think clearly, and showcase the developer you are becoming. You've got this!

Pro tip: Revise this guide once before each interview round. Focus on Part 4 (Interview Questions) the night before, and refresh Part 2 (CSS) and Part 3 (Bootstrap) which are heavily tested at the Associate Software Engineer level.